

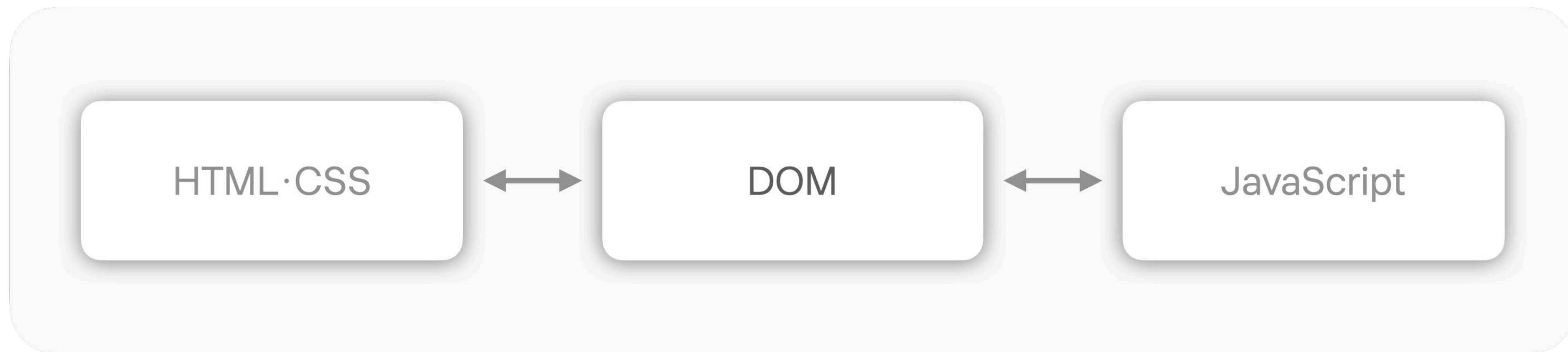
# 🔍 웹 브라우저의 생애주기

2026. 05. 15  
신수진

# 교육 목차

1. 문서 객체 모델 DOM
2. Virtual DOM
3. 컴포넌트 생명주기
4. 브라우저의 이벤트 흐름

# 문서 객체 모델 DOM

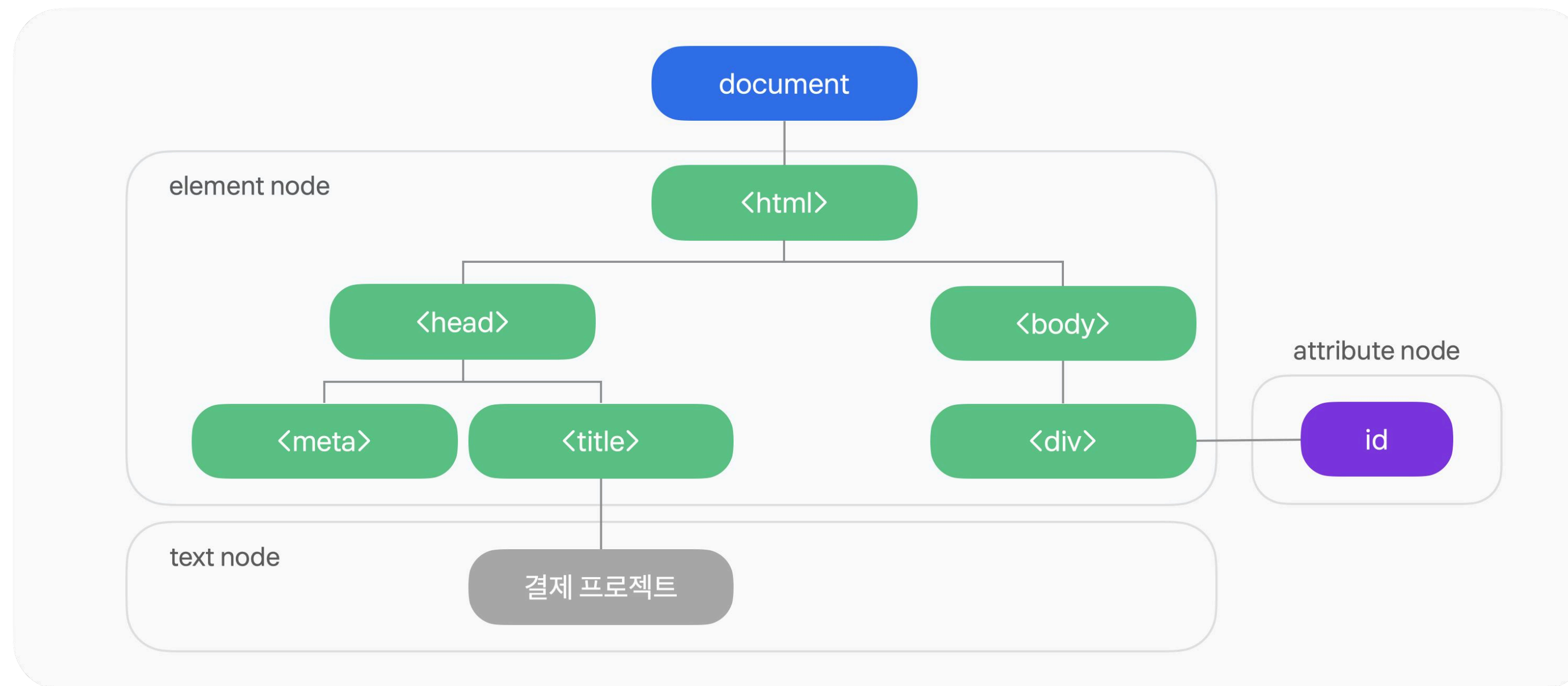


- DOM: HTML 요소의 계층을 반영해서 만든 객체
- JavaScript와 같은 스크립트 및 프로그래밍 언어가 웹페이지에 접근

DOM은 HTML로 구성된 웹 페이지와 스크립트 및 프로그래밍 언어를 연결시켜주는 역할



# DOM의 계층 구조



DOM의 계층은 트리 자료 구조를 가짐

# DOM에 접근하는 방법



```
document.getElementById(id);
```

주어진 ID와 일치하는 DOM 요소를 나타내는 Element객체  
문서 내에 주어진 ID가 없으면 null

# DOM에 접근하는 방법 - 예시

```
1 <html>
2   <head>
3     <title>getElementById 예제</title>
4     <script>
5       function changeColor(newColor) {
6         const elem = document.getElementById("para");
7         elem.style.color = newColor;
8       }
9     </script>
10  </head>
11  <body>
12    <p id="para">코테이토 화이팅</p>
13    <button onclick="changeColor('blue')">blue</button>
14    <button onclick="changeColor('red')">red</button>
15  </body>
16 </html>
```

## changeColor(newColor) 함수 설명

newColor로 선언된 파라미터 자리에 들어오는 값으로  
elem변수의 색상 스타일을 변경하는 함수

## document.getElementById("para")

document에서 선언된 노드들 중  
para라는 id를 갖고 있는 노드의 값을  
리턴하여 elem 변수로 선언



# DOM에 접근하는 방법 - 예시

```
1 <html>
2   <head>
3     <title>getElementById 예제</title>
4     <script>
5       function changeColor(newColor) {
6         const elem = document.getElementById("para");
7         elem.style.color = newColor;
8       }
9     </script>
10  </head>
11  <body>
12    <p id="para">코테이토 화이팅</p>
13    <button onclick="changeColor('blue')">blue</button>
14    <button onclick="changeColor('red')">red</button>
15  </body>
16 </html>
```



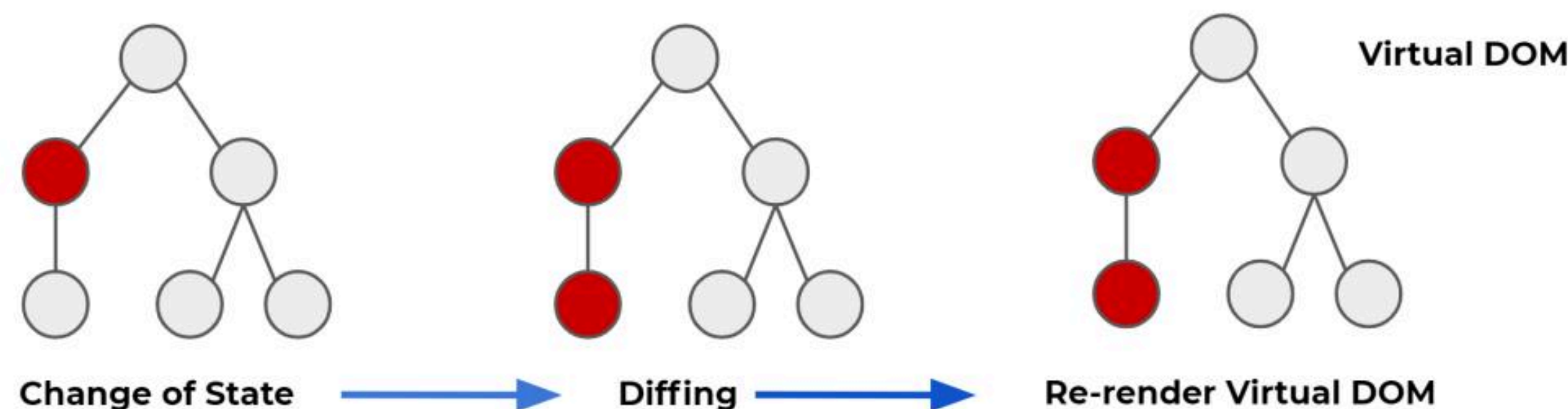
# DOM에 접근하는 방법 - 예시

1. blue버튼을 클릭
2. `changeColor('blue')` 함수 호출
3. `newColor` 파라미터에 `blue` 적용되어 함수 실행
4. `para`를 `id`로 갖고 있는 `element` 객체 찾기
5. 코테이토 화이팅의 색상 스타일이 `blue`로 변환





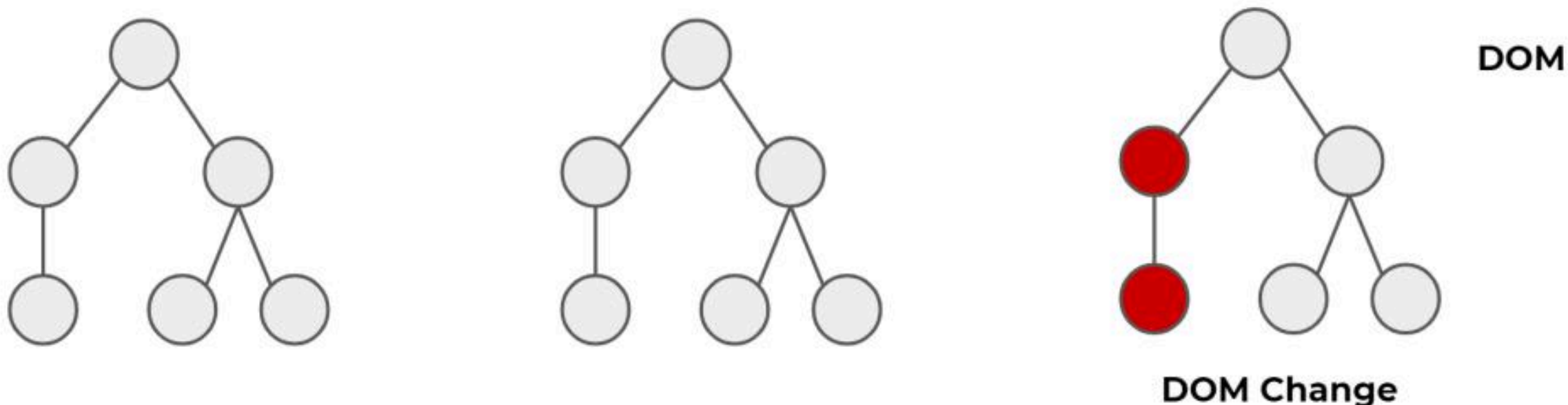
# Virtual DOM



Virtual DOM

## Diffing

빨간 부분에 수정사항이 생기게 되면  
Virtual DOM은 달라진 값을 탐지하여  
변경하고 최종적인 결과물을 실제 DOM에 전달

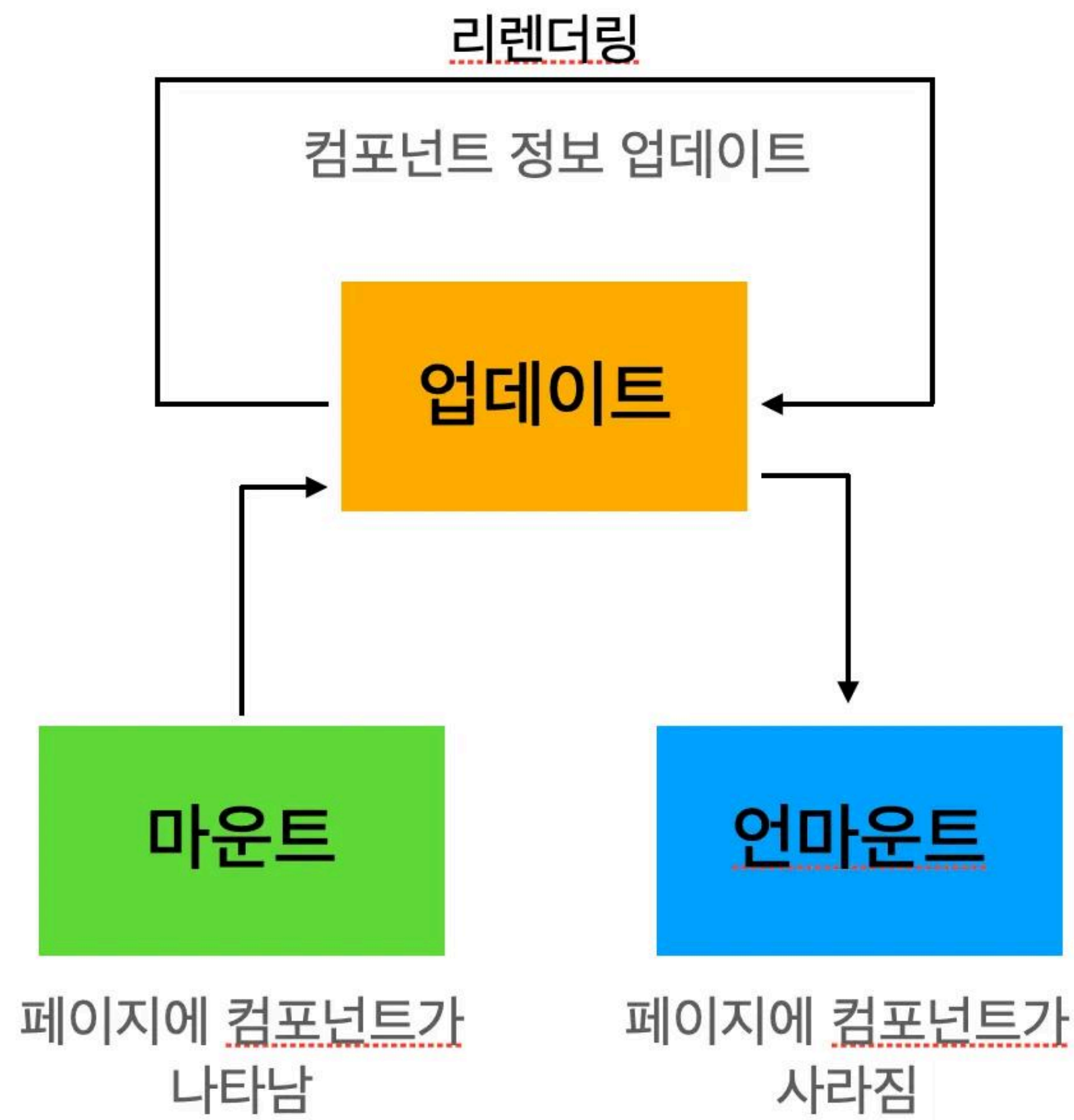


DOM

DOM Change

Virtual DOM이 없었더라면 DOM은 변경된 빨간 부분뿐만 아니라 모든 동그라미들을 빨간색으로 바꿔서 렌더링 했을 것  
→ 브라우저 과부하

# 컴포넌트 생명주기



## 마운트

컴포넌트가 처음 화면에 나타나는 순간입니다.  
이때 API를 호출하거나 이벤트 리스너를 등록하는 작업을 함

## 업데이트

상태나 props가 바뀔 때마다 컴포넌트가 다시 그려지는 단계

## 언마운트

컴포넌트가 화면에서 사라지는 순간 마운트 때 등록한 이벤트 리스너나 타이머를 제거. 제거하지 않으면, 컴포넌트는 사라졌지만 뒤에서 계속 동작하는 메모리 누수가 생김

# 컴포넌트 생명주기 - 예시

```
1  useEffect(() => {
2    // 1. [마운트] & [업데이트] 시점 실행
3    console.log(`사용자 ${userId}의 채팅 구독을 시작합니다.`);
4    const connection = chatAPI.subscribe(userId);
5
6    return () => {
7      // 2. [업데이트 직전] & [언마운트] 시점 실행
8      console.log(`사용자 ${userId}의 구독을 해지합니다.`);
9      connection.unsubscribe();
10   };
11 }, [userId]); // 의존성 배열에 userId가 들어감
```

**useEffect:** 컴포넌트가 렌더링 될 때마다 특정 작업을 실행하는 함수  
→ 유저가 바뀔 때만 실행됨  
카페 진동벨로 비유 하자면, 주문자가 바뀔 때만 진동벨 번호 갱신과 같은 개념

## 1. 마운트

- 컴포넌트가 처음 나타날 때 `userId`가 무엇이든 간에 일단 실행
- `subscribe(userId)` 실행

## 2. 업데이트

- `userId` 값이 변경될 때마다 발생합니다.
  - (예: 1번 유저 보다가 2번 유저 클릭 시)
- a. Cleanup 실행:
  - i. 먼저 이전 `userId`를 가지고 `return` 내의 `unsubscribe()`를 실행
- b. Effect 실행:
  - i. 그 다음 새로운 `userId`를 가지고 `subscribe()`를 실행

## 3. 언마운트

- 컴포넌트가 화면에서 사라질 때 마지막으로 정리 작업
- 현재 유지되고 있던 마지막 `userId`에 대해 `unsubscribe()`를 실행



# 컴포넌트 생명주기 - 예시

Q. 만약 다음과 같이 의존성 배열이 비어있다면?

```
1  useEffect(() => {  
2    // 1. [마운트] & [업데이트] 시점 실행  
3    console.log(`사용자 ${userId}의 채팅 구독을 시작합니다.`);  
4    const connection = chatAPI.subscribe(userId);  
5  
6    return () => {  
7      // 2. [업데이트 직전] & [언마운트] 시점 실행  
8      console.log(`사용자 ${userId}의 구독을 해지합니다.`);  
9      connection.unsubscribe();  
10   };  
11  }, []);
```

- 컴포넌트가 처음 나타날 때(Mount) 딱 한 번만 실행
- userId가 바뀌어도 새로운 채팅방을 구독하지 않음

카페 진동벨로 비유 시)

첫 손님 번호로 하루 종일 벨을 울림

다른 손님은 주문이 준비되어도 진동벨 알림을 받을 수 없음

# 컴포넌트 생명주기 - 예시

Q. 만약 다음과 같이 의존성 배열이 없다면?

```
useEffect(() => {  
  // 1. [마운트] & [업데이트] 시점 실행  
  console.log(`사용자 ${userId}의 채팅 구독을 시작합니다.`);  
  const connection = chatAPI.subscribe(userId);  
  
  return () => {  
    // 2. [업데이트 직전] & [언마운트] 시점 실행  
    console.log(`사용자 ${userId}의 구독을 해지합니다.`);  
    connection.unsubscribe();  
  };  
});
```

- 컴포넌트가 리렌더링될 때마다 실행
- 화면이 아주 조금이라도 바뀔 때마다 기존 구독을 끊고 새로 연결합니다 (글자를 입력하거나 다른 버튼을 누를 때마다)

카페 진동벨로 비유 시)

숨 쉴 때마다 진동벨 번호 다시 부여, 주방 서버 터짐 (성능 저하)



# 상태와 클로저

```
import React, { useState } from "react";

function LikeButton() {
  // 좋아요 갯수를 0이 초기화된 상태로 선언
  const [likes, setLikes] = useState(0);

  const handleLike = () => {
    // 사용자가 하트를 누르면, 'handleLike' 함수 실행
    // '기존에 하트가 몇 개였는지'를 기억해서 1을 더함
    setLikes(likes + 1);
  };

  return (
    <div>
      <h2>클로저 예시</h2>
      <button onClick={handleLike}>❤️ {likes}</button>
    </div>
  );
}
```

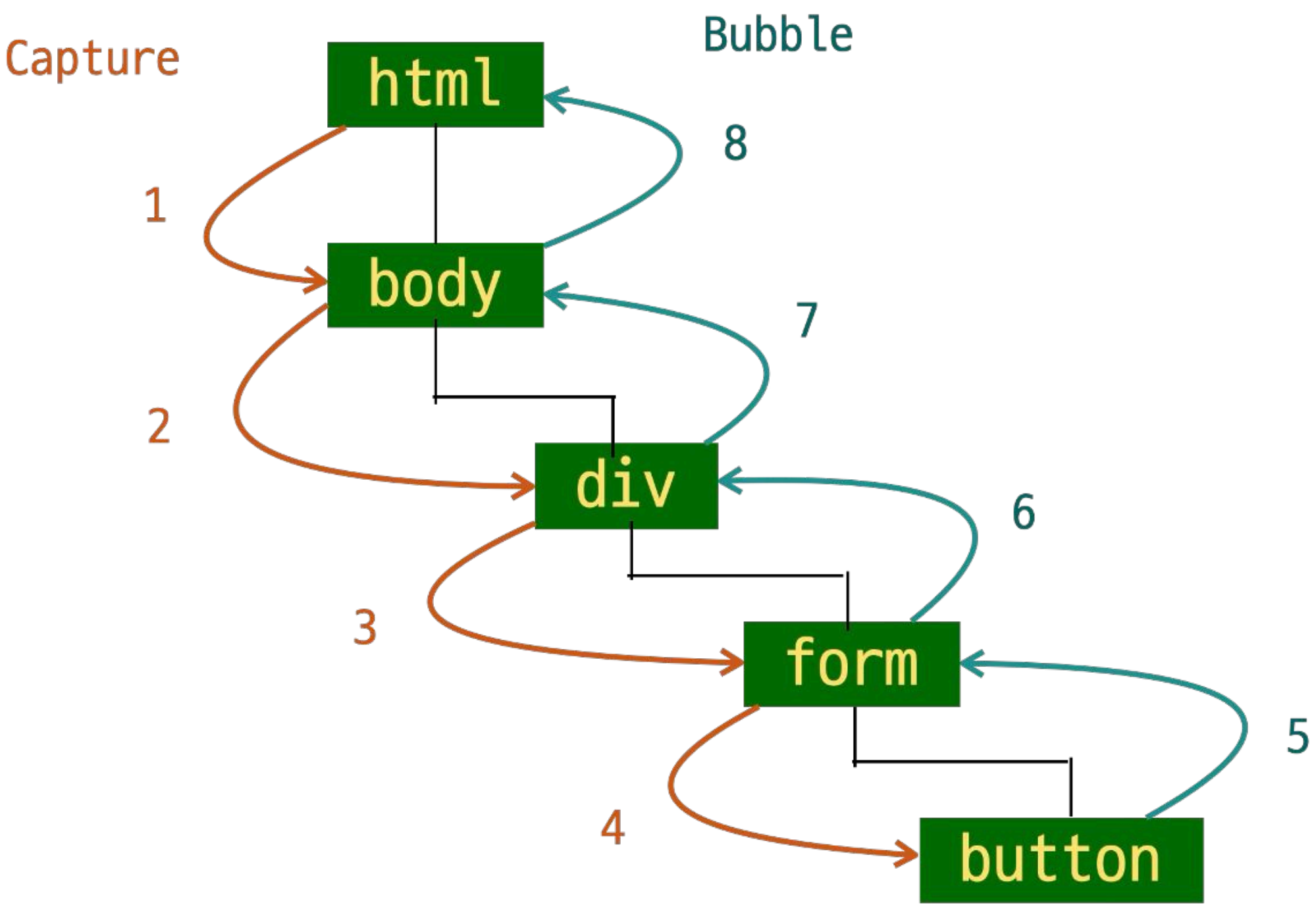
## 클로저

렌더링될 때마다 새로 실행되는 컴포넌트 함수가  
이전 상태를 기억하는 방법

리액트 함수 컴포넌트는 상태가 바뀔 때마다  
전체가 다시 실행되는 특징을 지님

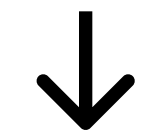
1. useState 호출 → 리액트 내부 메모리에 공간 확보.
2. setState 함수 생성  
→ 클로저 메커니즘을 통해 외부 메모리 공간에 대한 영구적 참조
3. 컴포넌트 재실행 → 클로저 덕분에 이전 상태값을 그대로 복원.

# 브라우저의 이벤트 흐름



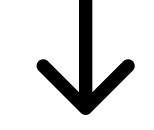
## 캡처링(capturing)

이벤트가 최상위 요소(document)에서 시작하여 실제 타깃 요소까지 하위로 내려감



## 타깃(target)

이벤트가 실제 발생한 요소에서 핸들러가 실행되는 시점



## 버블링(bubbling)

이벤트가 다시 타깃 요소에서 상위 요소로 역방향으로 올라가며 전파되는 단계

# 브라우저의 이벤트 흐름 - 예시

```
1 <html>
2   <body>
3     <div id="parent">
4       <button id="child">Click me</button>
5     </div>
6     <script>
7       document.getElementById("parent").addEventListener("click", () => {
8         console.log("부모 요소 클릭");
9       });
10
11      document.getElementById("child").addEventListener("click", () => {
12        console.log("자식 요소 클릭");
13      });
14    </script>
15  </body>
16 </html>
```

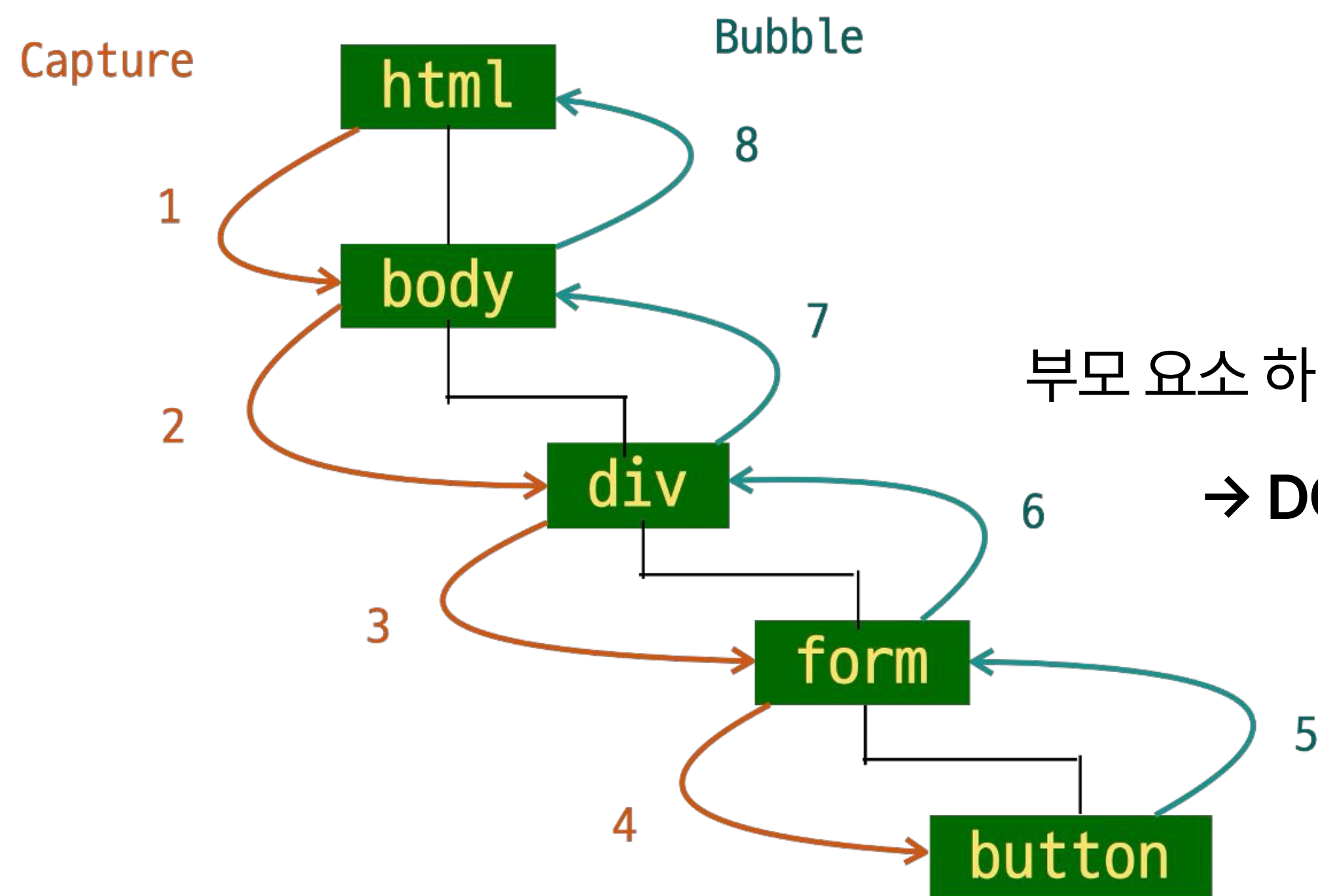
Click me

Elements Console Sources Network Performance  
top Filter  
자식 요소 클릭  
부모 요소 클릭  
> cmd i to turn on code suggestions. Don't show again

타겟에서 실행된 후 이벤트가 부모 요소로 전파되었음  
이처럼 이벤트는 특정 요소에만 국한되지 않고, 트리를 따라 이동하는 특징이 있음



# 이벤트 위임



많은 DOM 요소에 각각 이벤트를 등록하는 대신,  
부모 요소 하나에만 이벤트를 위임하면 성능과 유지보수 측면에서 훨씬 유리  
→ DOM 구조를 보다 효율적으로 설계할 수 있는 강력한 도구



# 이벤트 위임 - 예시

## 과일 목록 (이벤트 위임)

사과

바나나

오렌지

```
<h2>과일 목록 (이벤트 위임)</h2>
<ul id="itemList">
  <li>사과</li>
  <li>바나나</li>
  <li>오렌지</li>
</ul>
```

```
<script>
  const itemList = document.getElementById("itemList");

  itemList.addEventListener("click", function (event) {
    const targetLi = event.target.closest("li");

    if (targetLi && itemList.contains(targetLi)) {
      Array.from(itemList.children).forEach((li) =>
        li.classList.remove("active"),
      );
      targetLi.classList.add("active");
      console.log("선택된 항목:", targetLi.innerText);
    }
  });
</script>
```

# 이벤트 위임 - 예시

```
<script>
1 const itemList = document.getElementById("itemList");
  itemList.addEventListener("click", function (event) {
2   const targetLi = event.target.closest("li");
3   if (targetLi && itemList.contains(targetLi)) {
     Array.from(itemList.children).forEach((li) =>
       li.classList.remove("active"),
     );
4   targetLi.classList.add("active");
     console.log("선택된 항목:", targetLi.innerText);
   }
  });
</script>
```

## 1. 부모 요소 감시 (Event Listener)

- 개별 항목(li)이 아닌, 항목을 감싸는 부모 컨테이너(ui)에 단 하나의 이벤트 리스너를 등록

## 2. 클릭 타겟 파악 (Event Target)

- event.target을 통해 실제 클릭된 요소가 무엇인지 확인합니다.

## 3. 유효성 검사 (Filtering)

- .closest('li') 메서드를 사용하여 클릭된 지점이 리스트 항목 내부인지 판별합니다.

## 4. 동작 실행 (Action)

- 조건에 부합하는 경우에만 원하는 로직을 수행합니다.



# 이벤트 위임 - 예시

## 사과 선택

사과

바나나

오렌지

Elements Console Sources N

top Filter

선택된 항목: 사과

> cmd i to turn on code suggestions. Do

## 사과 선택 후 오렌지 선택

사과

바나나

오렌지

Elements Console Sources Netv

top Filter

선택된 항목: 사과

선택된 항목: 오렌지

# 이벤트 전파 흐름 제어

```
child.addEventListener("click", (e) => {  
  e.stopPropagation();  
  console.log("자식에서 전파 중단");  
});
```

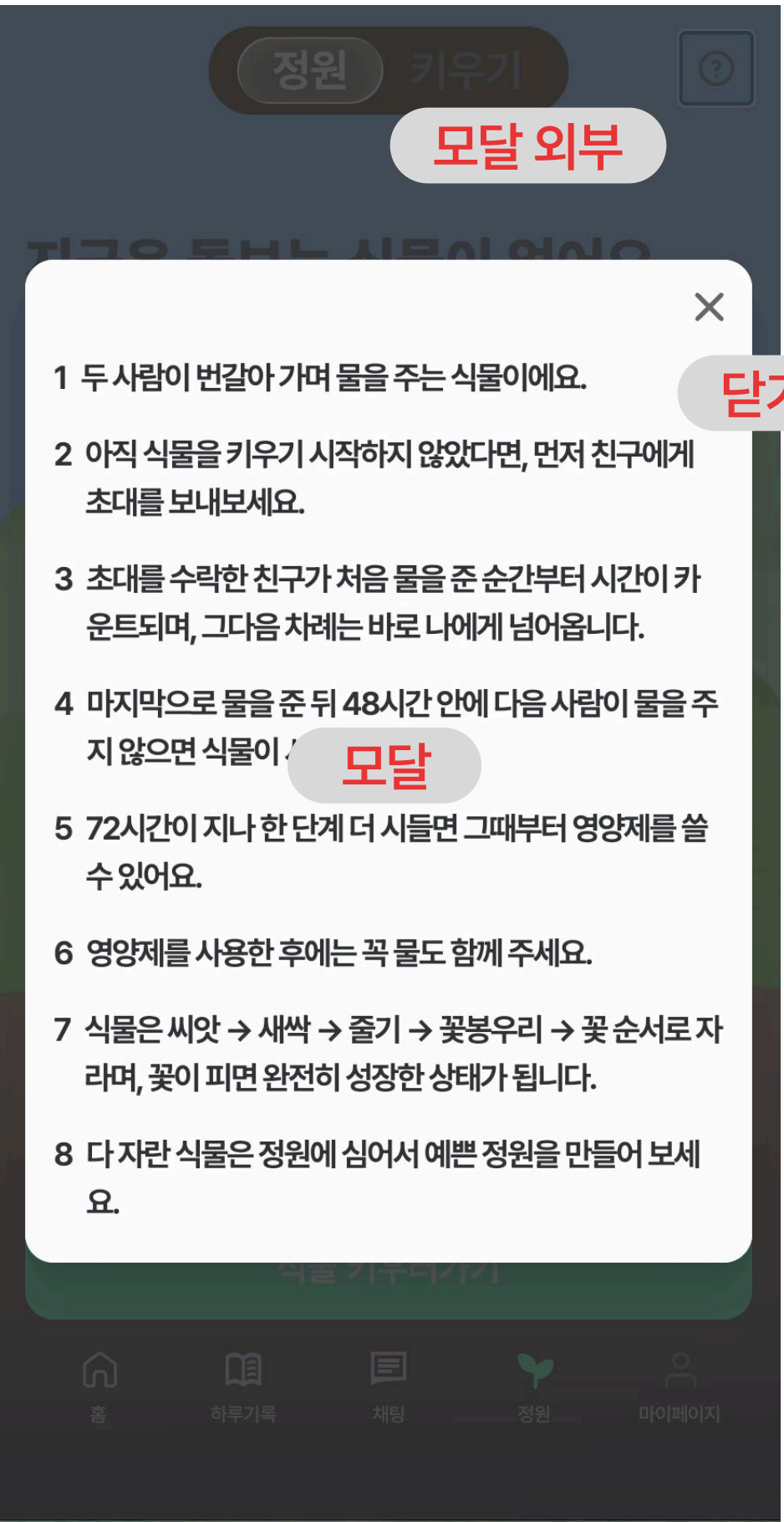
**event.stopPropagation()**

현재 이벤트 이후의 전파를 막는 역할을 함  
즉, 더 이상 상위 요소로 전달되지 않도록 차단하는 역할을 함  
→ 자식 요소의 핸들러만 실행되고, 부모 요소의 핸들러는 호출되지 않음



# 이벤트 전파 흐름 제어 - 예시

- 모달 바깥 영역에 onclick 이벤트 시에 closeModal() 함수를 통해 모달 닫기
- 모달 영역 내의 모달 닫기 버튼이 onclick 이벤트 시에 closeModal() 함수를 통해 모달 닫기

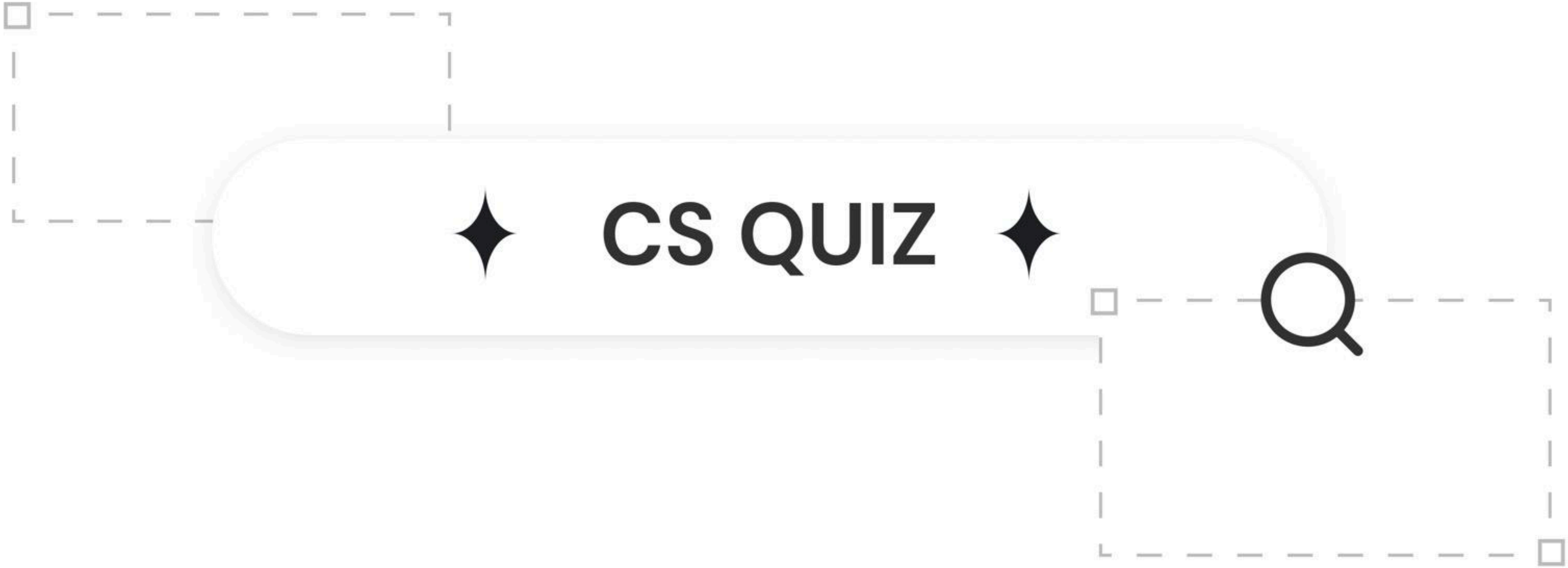


## event.stopPropagation() 적용 전

[모달 닫기 버튼]을 클릭 시, closeModal() 함수를 통해 모달에 대한 요소를 삭제하는데, 버블링을 통해 모달 바깥 영역에 적용된 closeModal() 함수가 다시 호출되면서, 이미 삭제한 모달요소를 다시 삭제하려니 에러가 발생

## [모달] 영역에 click 이벤트 시에 event.stopPropagation() 을 적용

[모달] 영역 밖에서 발생하는 이벤트 버블링을 막아주는 동시에 [모달] 내부에서 [모달 닫기 버튼] 영역을 제외한 부분을 클릭하면 모달이 닫히는 문제를 해결 가능



Q.1

**빈칸에 들어갈 알맞은 말을 쓰세요.  
DOM의 한글 명칭은? (00 00 00)**





A.1

DOM의 한글 명칭은? (00 00 00)

정답: 문서 객체 모델

DOM은 Document Object Model의 약자로  
한국어 명칭으로는 문서 객체 모델입니다.

Q.2

브라우저에서 이벤트가 발생했을 때 전달되는  
이벤트 전파(Event Propagation)의 3단계를 순서대로 나열하세요

답변 제출 예시) ㄱ, ㄴ, ㄷ

ㄱ. 버블링

ㄴ. 캡처링

ㄷ. 타깃



A.2

브라우저에서 이벤트가 발생했을 때 전달되는  
이벤트 전파(Event Propagation)의 3단계를 순서대로 나열하세요

ㄱ. 버블링

ㄴ. 캡처링

ㄷ. 타깃

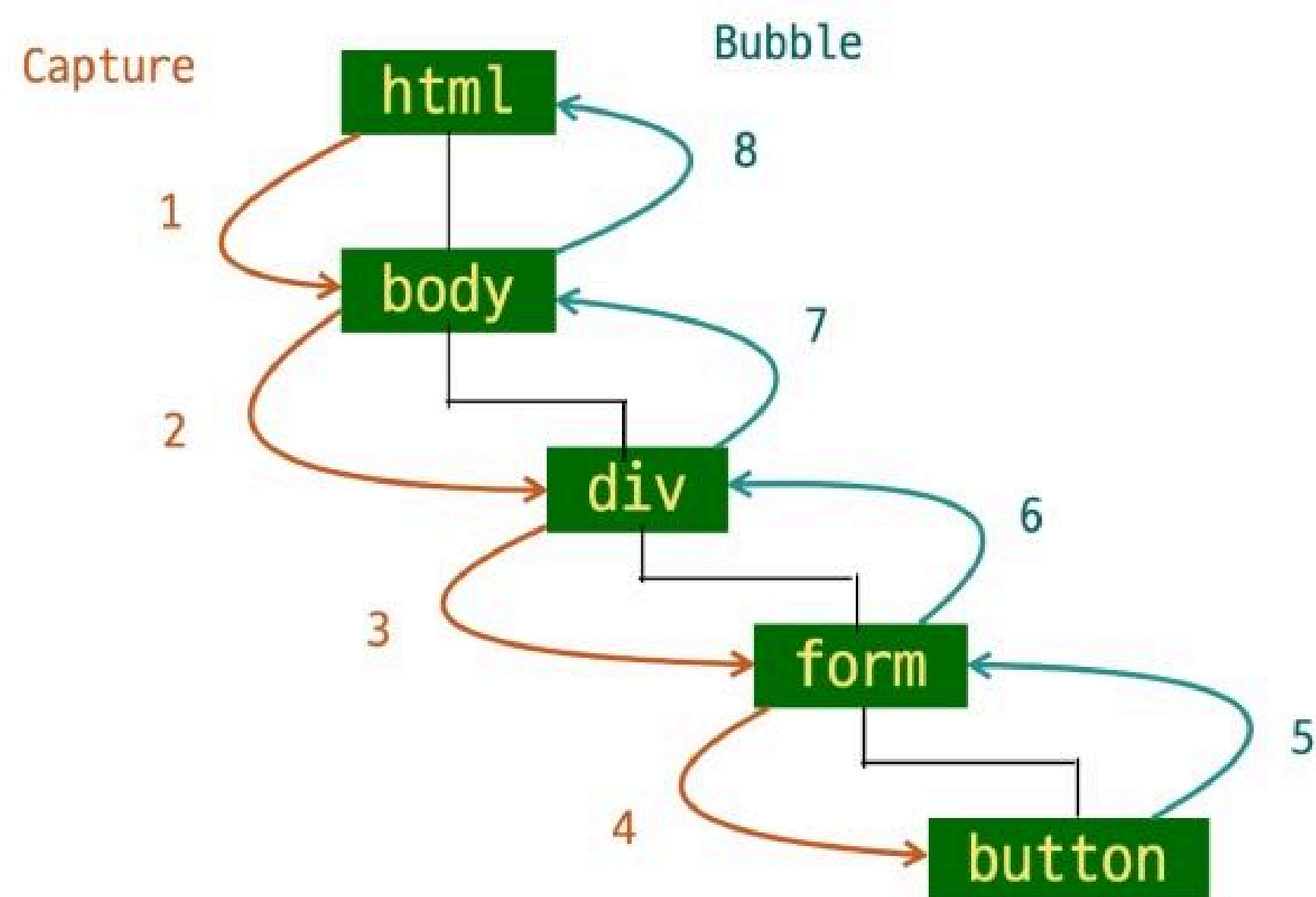
정답: ㄴ, ㄷ, ㄱ

브라우저의 이벤트는 단순히 발생한 지점에서 끝나는 것이 아니라,  
캡처링 → 타깃 → 버블링 순서의 3단계의 순차적인 과정을 거칩니다.

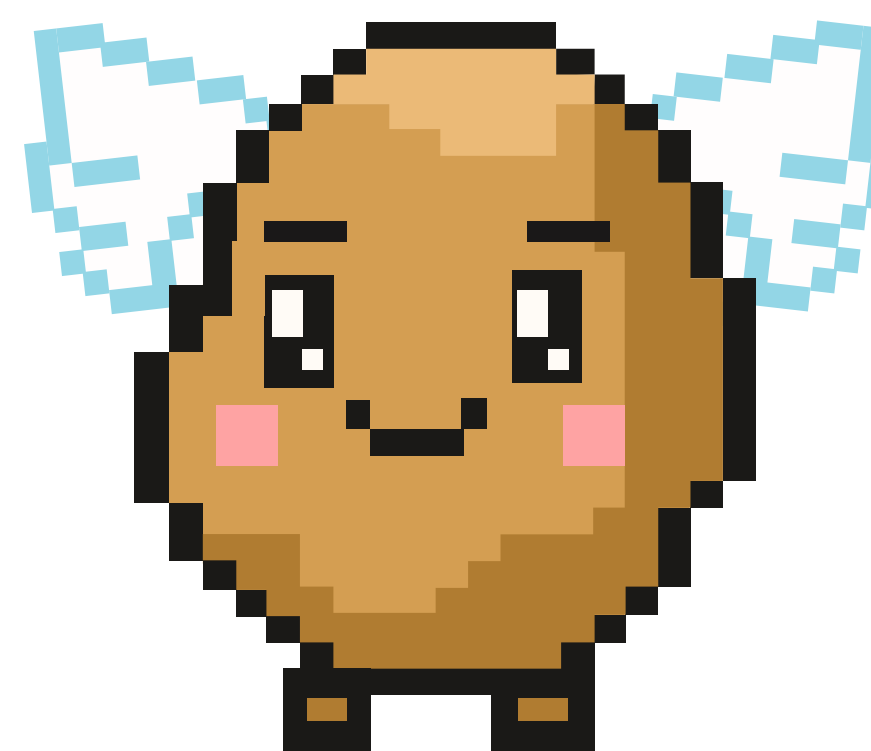


Q.3

브라우저에서 이벤트가 발생했을 때,  
아래 그림처럼 타깃 요소에서 시작해  
다시 부모 요소로 거슬러 올라가는 단계는?



거품이 위로 올라가는 모습과  
비슷해 붙여진 이름이야!





A.3

브라우저에서 이벤트가 발생했을 때,  
아래 그림처럼 타깃 요소에서 시작해  
다시 부모 요소로 거슬러 올라가는 단계는?  
(힌트: 거품이 위로 올라가는 모습과 비슷합니다.)

**정답 : 버블링 , bubbling, Bubbling**

이벤트는 캡처링(capturing) → 타깃(target) → 버블링(bubbling) 순서로 흐릅니다.  
대부분의 이벤트 처리는 이 '버블링' 단계에서 일어납니다.

Q.4

**virtualDom에서 DOM의 수정사항이 생겼을 때  
처음부터 다시 DOM을 그리지 않고,  
달라진 부분에 대해서만 조회하여 변경하는 기법은?**

- |                   |                          |
|-------------------|--------------------------|
| <b>1. Parsing</b> | <b>3. Rendering Only</b> |
| <b>2. Diffing</b> | <b>4. Mapping</b>        |



A.4

**virtualDom에서 DOM의 수정사항이 생겼을 때  
처음부터 다시 DOM을 그리지 않고,  
달라진 부분에 대해서만 조회하여 변경하는 기법은?**

- |                   |                          |
|-------------------|--------------------------|
| <b>1. Parsing</b> | <b>3. Rendering Only</b> |
| <b>2. Diffing</b> | <b>4. Mapping</b>        |

DOM에 수정사항이 생기게 되면 달라진 값을 탐지하여 변경하고  
최종적인 결과물을 실제 DOM에 전달하는 가상 돔의 핵심 기법입니다.



Q.5

**‘이벤트 위임’을 사용하면 모든 하위 항목에  
각각 이벤트 리스너를 등록해야 하므로  
메모리 사용량이 크게 늘어나는 단점이 있다.**

**( O , X )**





A.5

**‘이벤트 위임’을 사용하면 모든 하위 항목에  
각각 이벤트 리스너를 등록해야 하므로  
메모리 사용량이 크게 늘어나는 단점이 있다.**

**(0, X)**

개별 항목이 아니라 항목을 감싸는 부모 컨테이너에  
단 하나의 이벤트 리스너만 등록되기 때문에  
오히려 메모리 사용을 줄일 수 있어 성능과 유지보수 측면에서 훨씬 유리하다.

Q.6

자바스크립트로 DOM 요소에 접근할 때,  
특정 ID를 가진 요소를 찾기 위해 사용하는 메소드로  
가장 올바른 것은?

1. `document.getName()`
2. `document.getElementById()`
3. `document.findID()`
4. `document.selectNode()`



A.6

자바스크립트로 DOM 요소에 접근할 때,  
특정 ID를 가진 요소를 찾기 위해 사용하는 메소드로  
가장 올바른 것은?

- |                              |                          |
|------------------------------|--------------------------|
| 1. document.getName()        | 3. document.findID()     |
| 2. document.getElementById() | 4. document.selectNode() |

document의 요소들의 id의 값을 get하는 함수로  
찾고자 하는 특정 id를 파라미터에 넣어서  
해당 id를 갖고 있는 DOM요소에 접근할 수 있는 방법입니다.



Q.7

의존성 배열이 없는 useEffect의 실행 방식으로 올바른 것은?

1. 컴포넌트가 처음 마운트될 때 한 번만 실행된다.
2. userId가 변경될 때만 실행된다.
3. 컴포넌트가 리렌더링될 때마다 실행된다.
4. 언마운트 시에만 실행된다.





A.7

의존성 배열이 없는 useEffect의 실행 방식으로 올바른 것은?

1. 컴포넌트가 처음 마운트될 때 한 번만 실행된다.
2. userId가 변경될 때만 실행된다.
3. **컴포넌트가 리렌더링될 때마다 실행된다.**
4. 언마운트 시에만 실행된다.

화면 내, 컴포넌트 내용이 아주 조금이라도 바뀔 때마다 실행됩니다.  
(입력폼의 경우 글자를 입력될 때 마다) 이는 서버에 부하를 줄 수 있으므로,  
useEffect 사용 시에 의존성 배열을 꼭 추가해주셔야합니다.

## Q.8

**DOM(문서 객체 모델)에 대한 설명으로 옳지 않은 것은?**

1. HTML 요소의 계층 구조를 반영하여 만든 객체 모델이다.
2. 최상위 'document' 객체를 시작으로 하는 트리(Tree) 구조를 가진다.
3. 자바스크립트와 HTML을 연결해 주는 역할을 한다.
4. 리액트의 '가상 돔(Virtual DOM)'은  
매번 실제 DOM을 직접 수정하여 렌더링 효율을 높인다.



## A.8

DOM(문서 객체 모델)에 대한 설명으로 옳지 않은 것은?

1. HTML 요소의 계층 구조를 반영하여 만든 객체 모델이다.
2. 최상위 'document' 객체를 시작으로 하는 트리(Tree) 구조를 가진다.
3. 자바스크립트와 HTML을 연결해 주는 역할을 한다.
4. 리액트의 '가상 돔(Virtual DOM)'은  
매번 실제 DOM을 직접 수정하여 렌더링 효율을 높인다.

가상 돔은 실제 DOM을 직접 수정하는 것이 아니라,  
변화를 먼저 감지(Diffing)한 후 최종 결과만 실제 DOM에 전달하여 과부하를 줄인다.



Q.9

컴포넌트 생명주기에서 의존성 배열의 상태와 이를 카페 진동벨로 비유했을 때 발생하는 문제를 올바르게 연결하시오.

답변 제출 예시) 1-a, 2-b

(의존성 배열 상태 번호 먼저, 발생하는 문제 순서로 작성해주세요)

<의존성 배열 상태>

- 1. 의존성 배열이 비어있음
- 2. 의존성 배열이 존재하지 않음

<발생하는 문제>

- a. 숨 쉴때마다 진동벨 번호를 다시 부여해서 주방 서버가 터질 수 있다.
- b. 첫 손님 번호로 하루종일 벨이 울려서 다른 손님의 주문이 준비되어도 진동벨 알림을 받을 수 없다.





A.9

컴포넌트 생명주기에서 의존성 배열의 상태와 이를 카페 진동벨로 비유했을 때 발생하는 문제를 올바르게 연결하시오.

<의존성 배열 상태>

- 1. 의존성 배열이 비어있음
- 2. 의존성 배열이 존재하지 않음

<발생하는 문제>

- a. 숨 쉴때마다 진동벨 번호를 다시 부여해서 주방 서버가 터질 수 있다.
- b. 첫 손님 번호로 하루종일 벨이 울려서 다른 손님의 주문이 준비되어도 진동벨 알림을 받을 수 없다.

정답: 1-b, 2-a

- 의존성 배열이 비어있는 경우: 컴포넌트가 처음 나타날 때(Mount) 딱 한 번만 실행
  - 의존성 배열이 존재하지 않는 경우: 컴포넌트가 리렌더링 될 때마다 실행

Q.10

다음 중 컴포넌트 생명 주기에 대해 설명하는 학생 중  
**틀린** 설명을 하는 학생은?

- 다혜 '마운트'는 컴포넌트가 우리 눈에 처음 나타나는 단계야.  
이때 보통 API를 호출해서 데이터를 가져오거나 필요한 설정을 시작해.
- 철웅 '업데이트'는 컴포넌트의 상태(State)나 Props가 바뀌었을 때 발생해.  
데이터가 변하면 화면을 다시 그려주는 단계라고 보면 돼.
- 선아 '의존성 배열'이 비어있으면(`[]`), 컴포넌트의 상태값이 바뀔 때마다  
해당 코드가 매번 다시 실행되도록 설정한 거야.
- 세현 '언마운트'는 컴포넌트가 화면에서 사라지는 시점이야. 이때 사용하던  
이벤트 리스너를 지워줘야 메모리가 낭비되는 걸 막을 수 있어.



A.10

## 다음 중 컴포넌트 생명 주기에 대해 설명하는 학생 중 틀린 설명을 하는 학생은?

- 다혜 '마운트'는 컴포넌트가 우리 눈에 처음 나타나는 단계야. 이때 보통 API를 호출해서 데이터를 가져오거나 필요한 설정을 시작해.
- 철웅 '업데이트'는 컴포넌트의 상태(State)나 Props가 바뀌었을 때 발생해. 데이터가 변하면 화면을 다시 그려주는 단계라고 보면 돼.
- 선아 '의존성 배열'이 비어있으면(`[]`), 컴포넌트의 상태값이 바뀔 때마다 해당 코드가 매번 다시 실행되도록 설정한 거야.
- 세현 '언마운트'는 컴포넌트가 화면에서 사라지는 시점이야. 이때 사용하던 이벤트 리스너를 지워줘야 메모리가 낭비되는 걸 막을 수 있어.

의존성 배열이 비어있는 경우에는 컴포넌트가 처음 나타날 때(마운트) 딱 한 번만 실행됨. 값이 바뀔 때마다 매번 실행되게 하려면 의존성 배열 자체를 생략해야 함.